AMGHAR Massinissa
NGUYEN Alban

BACKPACK HERO

Manuel développeur

2025/2026

Table des matières

1. Introduction……..
2. Organisation générale…….
3. Package application……
4. Package character……...
5. Package effect……
6. Package game……
7. Package graphics……..
8. Package button……..
9. Package image……...
10. Package View…….
11. Package Interaction……..
12. Package inventory……
13. Package Map……..
14. Améliorations depuis la soutenance……

1. Introduction

Ce manuel décrit l'architecture logicielle de notre implémentation du jeu **Backpack Hero**. Il présente l'organisation des packages, le rôle des classes principales ainsi que les choix de conception effectués.

2. Organisation générale

Les sources du projet sont regroupées dans le répertoire **src/fr/uge**.

Le code est organisé en plusieurs packages, chacun correspondant à une responsabilité précise.

3. Package application

Le package **fr.uge.application** contient les classes responsables du lancement de l'application. Il contient notamment :

- **Main** : initialise l'interface graphique, charge les ressources du jeu et crée le contrôleur, avant de lancer la boucle principale qui exécute le jeu.
- **HallOfFame** : gère l'enregistrement et l'affichage des meilleurs scores.

Ces classes ne contiennent pas de logique complexe. Elles délèguent la logique du jeu aux autres packages.

4. Package character

Le package **fr.uge.character** regroupe toutes les entités du jeu pouvant interagir avec le héros. Il contient notamment les classes :

- **Hero** : représente le personnage incarné par le joueur.
- **Enemy** : représente un ennemi individuel.
- **Enemies** : permet de créer un groupe d'ennemis.
- **Merchant** : représente le marchand.
- **Healer** : représente le guérisseur

L’interface **Fighter** permet de regrouper les actions que le héros et les ennemis ont en commun, tout en proposant une implémentation spécifique.

Afin d’éviter de surcharger la classe **Enemy** avec les données statiques ou purement descriptives, nous avons créé le record **EnemyData**. Ce record regroupe l’ensemble des caractéristiques immuables d’un ennemi, telles que son nom, ses points de vie maximum, ses plages d’attaque et de défense, ainsi que l’expérience qu’il rapporte. La classe **Enemy** se concentre alors uniquement sur le comportement et l’état dynamique de l’ennemi durant la partie. Ce choix de conception permet de mieux séparer les responsabilités, d’améliorer la lisibilité du code et de faciliter l’ajout de nouveaux types d’ennemis.

La classe **Hero** est une façade. L’ensemble des actions et des informations du héros est accessible depuis cette classe, ce qui simplifie son utilisation dans le reste du code. En revanche, elle ne contient pas directement l’implémentation de ces actions, afin d’éviter une classe trop volumineuse et peu lisible. La logique interne a ainsi été répartie dans plusieurs sous-classes en fonction de leurs responsabilités. La classe **HeroFight** regroupe tout ce qui concerne les mécaniques de combat et l’état du héros. La classe **HeroProgression** gère la progression du héros, notamment l’expérience et les montées de niveau. Enfin, la classe **HeroInventoryManager** est responsable de la gestion de l’inventaire, en particulier de la possibilité d’achat d’un item. Cette organisation permet d’ajouter de nouvelles méthodes très facilement.

5. Package effect

Le package **fr.uge.effect** regroupe les effets appliqués par les objets lors des combats. Il définit une interface commune, **ItemEffect**, permettant d’unifier le comportement des différents effets. Elle est implémentée par les classes suivantes : **PoisonEffect**, **PoisonArrowEffect**, et enfin, **DamagePerTurnEffect**. Ce package permet d’ajouter facilement de nouveaux effets sans modifier la logique de combat existante.

6. Package effect

Le package **fr.uge.game** regroupe les classes responsables du déroulement du jeu et la gestion des différentes phases de gameplay. Il contient notamment les classes :

- **FightGraphics** : gère le déroulement des combats, l'alternance des tours, l'application des effets, les actions des ennemis et les conditions de fin de combat.
- **GameGraphics** : initialise l'état général du jeu, notamment le donjon, le héros, le sac à dos et le premier étage.
- **ManageBackpack** : gère les interactions avec le sac à dos.

7. Package graphics

Le package **fr.uge.graphics** regroupe une classe et un record, responsables de la gestion de l'interface graphique et des intéractions avec l'utilisateur :

- **Controller** : gère le déroulement du jeu (boucle principale, transitions entre les différentes vues).
- **Message** : représente un message temporaire affiché à l'écran.

Ce package assure le lien entre la logique du jeu et son interface graphique.

8. Package button

Le package **fr.uge.graphics.button** regroupe les classes représentant les différents boutons cliquables du jeu. Ces classes gèrent l'affichage des boutons, la détection des clics et les actions associées. Il contient notamment les classes :

- **Button** : fournit l'affichage et la détection de clic d'un bouton graphique.
- **DeleteButton** : pour supprimer un item.
- **EndTurnButton** : pour terminer son tour lors d'un combat.
- **HealButton** : pour récupérer des points de vie chez le guérisseur.
- **RefuseButton** : pour refuser une malédiction lors d'un combat.

- **SellButton** : pour vendre un item chez le marchand.
- **RotateButton** : pour faire tourner un item à 90 degrés.

9. Package image

Le package **fr.uge.graphics.image** contient la classe **ImageLoader** qui permet de charger et stocker des images depuis des fichiers, puis à les dessiner avec mise à l'échelle sur l'écran via Graphics2D.

10. Package view

Le package **fr.uge.graphics.view** contient l'interface **View**, qui est implémentée par toutes les classes représentant l'affichage graphique du jeu. Elle définit les méthodes essentielles pour dessiner la vue à l'écran et réagir aux clics de l'utilisateur. Elle contient, entre autres, les classes **MapView**, **HeroView**, **HealerView** et **MerchantView**.

11. Package interaction

Le package **fr.uge.interaction** contient l'interface **ItemInteraction**, qui définit le comportement des intéractions spéciales entre les objets du héros et les ennemis lors des combats. Chaque classe implémentant cette interface précise un effet particulier déclenché par la présence ou la position d'objets spécifiques dans le sac du héros ou par d'autres conditions. Elle contient, entre autres, les classes **AdjacentArmorBlockInteraction**, **CurseBladeInteraction** et **GoldCLeaverInteraction**.

12. Package inventory

Le package **fr.uge.inventory** regroupe l'ensemble des classes et structures permettant de gérer les objets et l'inventaire du héros. Il contient notamment les classes :

- **Backpack** : représente le sac à dos du héros.
- **Item** : représente les objets du jeu.

Les classes **Backpack** et **Item** sont des façades. Elles donnent accès à toutes leurs données et leurs actions, ce qui simplifie leur utilisation dans le reste du code.

Afin de ne pas surcharger **Backpack**, nous avons créé **BackpackPlacement,** gérant la validation et l'ajout des objets sur la grille, tandis que **BackpackSearch** permet de rechercher facilement un objet spécifique selon son type ou ses caractéristiques. Cette organisation permet de séparer clairement les responsabilités, de rendre le code plus lisible et de faciliter l'ajout de nouvelles fonctionnalités, comme des objets spéciaux.

Pour éviter de surcharger la classe **Item** avec des données statiques ou purement descriptives, nous avons créé le record **ItemData**, qui regroupe les informations immuables d'un objet. Pour gérer des fonctionnalités spécifiques, plusieurs classes et structures auxiliaires ont été ajoutées. **ItemRotation** s'occupe de la rotation logique des objets. **ItemRegistry** centralise la création des objets et l'ajout automatique de leurs effets et interactions. **ItemShapes** fournit des modèles de formes pour les objets. Le package utilise également des enums, comme **ItemType** et **ItemRarity** pour donner respectivement le type et la rareté d'un objet.

13. Package Map

Le package **fr.uge.map** regroupe toutes les classes responsables de la structure du donjon et de la navigation du héros dans le jeu. Il contient notamment les classes suivantes :

- **Dungeon** : représente l'ensemble du donjon. Il gère la liste des étages, l'étage actuellement exploré et permet de passer d'un étage au suivant. Il est également responsable de la création initiale du donjon.
- **Floor** : représente un étage du donjon sous forme d'une grille de salles. Cette classe gère le placement du héros, la génération des salles et des couloirs, les déplacements possibles ainsi que la

vérification de l’accessibilité entre deux salles. Elle assure également la génération des salles et la connexion entre les différentes pièces.
- **Room** : représente une salle individuelle du donjon.

14. Améliorations depuis la soutenance

Depuis la soutenance, nous avons réalisé plusieurs améliorations majeures sur le projet. Lors de celle-ci, nous ne disposions pas encore d’une version graphique fonctionnelle : seule une version terminale était opérationnelle. Elle implémente l’ensemble des fonctionnalités de base du jeu et respecte intégralement les critères de la phase 1. Depuis, nous avons transposé cette version terminale vers une version graphique complète. À la demande de l’enseignant, nous avons refactorisé la classe Item, qui gérait auparavant trop de responsabilités, afin d’améliorer la lisibilité du code.

Plus généralement, nous avons procédé à un important travail de refactorisation sur plusieurs classes, supprimé celles qui n’étaient utiles que pour la version terminale et réduit la taille de certaines méthodes pour rendre le code plus clair.

Nous avons également ajouté une zone de suppression permettant au joueur d’abandonner des objets, lesquels restent alors dans la salle correspondante. Une fois la phase 1 implémentée, nous avons finalisé la phase 2 en intégrant l’expérience, la montée de niveau, les interactions entre équipements, ainsi que les effets de combat et les malédictions. Nous avons enfin bien entamé la phase 3, en implémentant le calcul d’un score, d’un Hall of fame, de nouveaux ennemis. Cependant, nous n’avons pas eu le temps de générer une map aléatoire et le plan des étages est encore codé en dur.